Intrinsic Preferences of AI Coding Agents: Thematic Priors, Fixation, and Defaults Under Underspecified Prompts

Changkun Ou [1 2]

Abstract

What do AI coding agents build when given no specification? Motivated by a production deployment where an autonomous pipeline plateaued into micro-optimizations, we conducted a controlled study across 405 sessions spanning 15 models, 3 providers, 2 backends, 5 prompts, and 2 harness versions. We document three phenomena: (1) large factor sensitivity, prompt wording determines whether agents propose or implement, whether outputs are 30 or 600 LOC, and whether targets are terminal or browser; (2) stable behavioral attractors, models exhibit persistent topical fixations with opus-4.6 choosing Game of Life in 10 of 10 runs and GPT models defaulting to todo apps; (3) thematic divergence under minimal prompting, the volitional prompt "Just do something you want" concentrates preferences onto distinct classical-CS touchstones, opus-4.6 toward self-reference (Game of Life, ray tracing) and opus-4.7 toward emergence (Mandelbrot, flocking, reaction-diffusion). These findings suggest conventional benchmarks may not capture decision-making behavior in underspecified regimes where agents are increasingly deployed. We release all experimental infrastructure and data.

1. Introduction

Autonomous AI coding agents such as Devin (Cognition, 2024) and coding assistants like Claude Code (Anthropic, 2025a), OpenAI Codex CLI (OpenAI, 2025), and Cursor (Anysphere, 2024), now resolve over 80% of real-world GitHub issues on SWE-bench Verified (Scale AI, 2026; Jimenez et al., 2024; Yang et al., 2024), with the Stanford AI Index reporting agent task success jumping from 12% to 66% in a single year (Stanford HAI, 2026). These results come from *well-specified* tasks: fix this bug, implement this feature, pass this test suite. Increasingly, however, agents are deployed in regimes where the task is under-specified: long-running autonomous loops, vague user requests, ambient agents acting in the background. What an agent *chooses* to do in such regimes is not directly measured by current benchmarks.

Our motivation is practical. We deployed a four-stage autonomous pipeline (strategist, executor, tester, documenter) on a production codebase for 10 days. Output was initially impressive, 627 commits, growing from 25K to 122K LOC, then plateaued into a *micro-optimization spiral* of locally optimal, globally irrelevant refinements (Christensen, 1997; Simon, 1956). Ablating the pipeline to a single agent revealed model-level defaults (Claude $\rightarrow$ Game of Life, GPT $\rightarrow$ todo apps) that were reproducible but tangled with prompt wording and environment context. We could not tell, from the deployment alone, how much of what we saw was a property of the model versus an artifact of the setup. So we ran a controlled sweep.

This paper is an empirical study of what AI coding agents produce under underspecified prompts. Across 405 sessions spanning 15 models, 2 backends, 5 prompts, and 2 harness versions, we report three observations:

(1) Factor sensitivity is large. Prompt wording determines whether models propose or implement; whether outputs are 30 or 600 LOC; whether targets are terminal or browser; whether runs fixate or diversify. Environment artifacts shift language selection from polyglot to Python-dominant. Harness version modulates fixation strength and complexity independently of the model.

(2) A volitional prompt concentrates the signal. A one-sentence directive ("Just do something you want."), stripped of project reference and goal framing, produced the sharpest preference signal in the study: opus-4.6 chose Conway's Game of Life in 5 of 5 runs; opus-4.7 chose the Mandelbrot set in 5 of 5 runs; implementations converged to near-identical LOC and structure within each model.

(3) Sibling models split cleanly on theme. Both Opus preferences are classical-CS touchstones that express complex behavior from simple rules, but they fall on different sides of a legible axis: opus-4.6 toward

[1]LMU Munich, Germany [2]Sixt SE, Germany. Correspondence to: Changkun Ou <research@changkun.de>.

self-reference (cellular automata, ray tracing, typing tests), opus-4.7 toward *emergence* (flocking, reaction-diffusion, maze and dungeon generation).

We report these as empirical observations, not as a theory of model priors or a proposed benchmark. The $n = 5$-per-cell design and single-point measurement on each model mean numerical rates are suggestive rather than conclusive, and we enumerate throughout the paper the robustness checks (paraphrase stability, temperature variation, functional verification) required to move from observation to claim.

2. Related Work

Code generation benchmarks. HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021) evaluate function-level code generation from docstrings. SWE-bench (Jimenez et al., 2024) scales to repository-level bug fixes. More recent benchmarks push further: ABC-Bench (Zhang et al., 2026) requires agents to manage the full backend development lifecycle across 8 languages and 19 frameworks, and NL2Repo-Bench (Xin et al., 2025) evaluates long-horizon repository generation from natural language descriptions. SWE-bench Pro (Scale AI, 2026) addresses contamination concerns with 1,865 multi-language tasks where models scoring 80%+ on Verified only reach 46–57%. All these benchmarks share one property: a well-specified task with a ground-truth solution. Our work complements them by removing the specification entirely, measuring what agents do when the "what to build" decision is theirs.

Autonomous agent loops. AutoGPT (Significant Gravitas, 2023) and BabyAGI (Nakajima, 2023) pioneered open-ended autonomous agent loops, where an LLM iteratively proposes and executes tasks toward a high-level objective. In practice, these systems frequently enter infinite loops, spiral into costly API calls, and struggle to determine termination conditions. More recent frameworks like SAGA (Lu et al., 2025) employ bi-level architectures where outer-loop agents evolve objectives while inner loops optimize under them, and CORAL (Human-Agent Society, 2026) replaces rigid control with persistent memory and asynchronous multi-agent collaboration. Our production deployment exhibits many of the same failure modes reported in these systems, but we go further by systematically isolating the factors that drive them.

Agent reliability and behavioral collapse. Recent work on long-horizon agents has documented *meltdown behavior*, where agents transition from coherent to incoherent looping over extended task horizons (Li et al., 2026). Aggregate pass@1 drops from 76.3% at short horizon to 52.1% at very long horizon, a 24-point decline. Our micro-optimization spiral is a related phenomenon at a longer timescale: the agent does not become incoherent, but its decisions become increasingly locally optimal and globally irrelevant.

Prompt sensitivity. LLM outputs are sensitive to prompt phrasing (Lu et al., 2022; Zhao et al., 2021). Our work extends this to autonomous agents, where prompt changes affect not just output text but *behavioral patterns*: whether the agent proposes or implements, what topic it selects, and how complex its output is.

Model behavioral tendencies. Studies of LLM biases (Santurkar et al., 2023) have shown that models exhibit consistent preferences. Recent work demonstrates that LLMs display human-like social desirability biases in personality assessments (Simonds et al., 2025) and develop emergent social conventions in multi-agent populations (Chuang et al., 2025). These findings suggest that "personality-like" profiles in LLMs are reproducible yet context-dependent. We document an analogous phenomenon in coding agents: persistent topical fixations that survive across prompt variations and repeated runs, and that differ across model versions in ways consistent with stable thematic priors.

Text degeneration and repetition. Neural text generation is prone to degenerate repetition, where models enter loops producing the same tokens or phrases (Holtzman et al., 2020). The fixation we observe operates at a higher level of abstraction: the agent does not repeat tokens but repeatedly *chooses the same project*. This is closer to mode collapse in generative models than to token-level degeneration, suggesting that the decision of what to build has a narrower effective distribution than the space of possible implementations.

Bounded rationality and satisficing. Simon's theory of bounded rationality (Simon, 1956) argues that agents with limited computational resources settle for satisfactory rather than optimal solutions. Christensen's innovator's dilemma (Christensen, 1997) describes how incumbents over-optimize along existing trajectories. We observe both patterns: without external disruption, agent systems satisfice within their initial framing, producing incremental improvements that crowd out structurally novel alternatives.

3. Experiment Setup

Our investigation proceeds in two stages: a production deployment with ablation studies (§3.1), and controlled single-session experiments (§3.2).

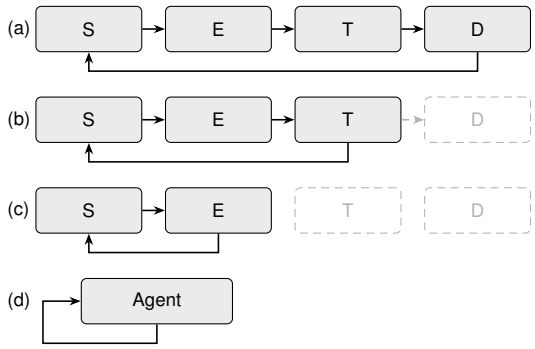

Figure 1. Ablation configurations. S = Strategist, E = Executor, T = Tester, D = Documenter. (a) Full pipeline. (b) Without Documenter: lost memory. (c) Without Tester: unchecked regressions. (d) Single agent: reveals model priors.

Table 1. Models tested across three provider families.

Provider	Model ID	Notes
Anthropic	claude-opus-4.7	Latest Opus (Exp4–7)
	claude-opus-4.6	Previous largest
	claude-opus-4.5	Previous generation
	claude-sonnet-4.6	Mid-tier, current gen
	claude-sonnet-4.5	Mid-tier, previous gen
	claude-haiku-4.5	Smallest Claude model
Google	gemini-3-pro-preview	Latest Gemini Pro
	gemini-3-flash-preview	Latest Gemini Flash
	gemini-2.5-pro	Previous generation
	gemini-2.0-flash	Older generation
OpenAI	gpt-5.4	Latest, largest
	gpt-5.1	Latest, mid-tier
	gpt-5-mini	Latest, small
	gpt-4.1	Previous generation
	gpt-4.1-mini	Previous, small

3.1. Production Deployment and Ablation

We summarize the production deployment that motivated the study; full details and logs are in the open-source repository. A four-stage agent pipeline (Strategist, Executor, Tester, Documenter, looping autonomously) operated on a production codebase for 10 days (March 7–16, 2026), producing 627 commits and growing the codebase from 25K to 122K LOC, before plateauing into the *micro-optimization spiral* described in §1.

Systematic ablation (Figure 1) isolated which pipeline components were load-bearing and, relevant for what follows, revealed model-specific defaults under the single-agent configuration: Claude (Opus 4.6) consistently chose Game of Life; GPT models defaulted to todo list applications. Without the Documenter, the Strategist lost historical context and reproposed completed work. Without the Tester, the system grew a single Game of Life implementation to 112K LOC in one file, then broke 12K lines in a refactor with no mechanism to detect the regression. Single-agent defaults were stable across repetitions but tangled with environment context and prompt wording, motivating the controlled experiments in §3.2.

3.2. Controlled Single-Session Experiments

We test 15 models across three providers (Table 1), accessed through a unified API gateway routing to Anthropic API, Google Vertex AI, and Azure OpenAI. Two containerized sandbox backends provide execution environments: the **Claude backend** (`sandbox-claude`) runs Claude Code with a full Linux environment; the **Codex backend** (`sandbox-codex`) runs OpenAI's Codex CLI with bubblewrap isolation. Non-native models are routed through litellm for API translation. Each session starts with a fresh, empty workspace. Environment bias is controlled by disabling pre-installed tools (specifically RTK (RTK AI, 2025)) via a no-op stub.

Five prompts progressively reduce task specification. Prompts 1–4 retain some form of project reference or imperative; Prompt 5 strips both: **Prompt 1** (Baseline): "Look at this project and decide on your own what to build, and DO it." **Prompt 2** (Goal-focused): adds "propose exactly ONE goal" and "pick a concrete, interesting idea." **Prompt 3** (Implementation demand): adds "JUST DO IT" after Experiment 2 revealed that some models proposed without implementing. **Prompt 4** (Bare imperative): "Build something. Just do it.," the implementation demand with project reference removed. **Prompt 5** (Volitional): "Just do something you want.," redirects topic choice from "what is appropriate to build" to "what the model prefers," and is the probe whose output we focus on in §5. Prompt 1 was used with RTK inadvertently present; Prompts 2–5 ran with RTK disabled.

Execution matrix. Experiment 1: 14 models $\times$ 2 backends $\times$ 5 runs = 140 sessions. Experiment 2: Claude backend only (70). Experiment 3: both backends (140). Experiment 4: opus-4.7 only, same prompt as Exp3, harness 2.1.109 (5). Experiment 5: opus-4.6 and opus-4.7, upgraded harness 2.1.112 (10). Experiment 6: opus-4.6 and opus-4.7 with Prompt 4 on harness 2.1.112 (10). Experiment 7: opus-4.6 and opus-4.7 with Prompt 5 (10), plus a supplementary replication of Prompt 3 on harness 2.1.112 for the other four Claude models (sonnet-4.6, sonnet-4.5, opus-4.5, haiku-4.5; 20 sessions). Total: 405 sessions (Figure 2).

Metrics. For each session we record: topic (manually classified), tech stack, complexity (files, LOC, function count), engineering maturity (tests, README, build config, CI), and runtime (exit code, wall-clock duration). Sessions with technical errors (exit $\neq 0$) are excluded from complexity metrics but included in topic/fixation analysis when partial output reveals the model's intent.

Figure 2. Controlled experiment design. Each experiment was informed by findings from the previous one. Total: 405 sessions across 15 models. CC = Claude Code. Experiment 7 includes 10 primary sessions (opus-4.6/4.7 on Prompt 5) plus 20 supplementary sessions replicating Prompt 3 for the other four Claude models on harness 2.1.112.

Table 2. Experiment 1 (Claude backend). N = sessions without technical error. RTK present, biased toward dev tooling.

Model	N	Avg LOC	Lang	Typical Project
claude-sonnet-4.5	5	776	Python	Dev workflow tools
claude-opus-4.6	5	607	Go/Rust	TUI apps
claude-sonnet-4.6	5	506	Python	Git/dev tools
claude-haiku-4.5	5	233	Py/Go/TS	Developer utilities
claude-opus-4.5	5	221	Go/JS/Py	CLI utilities
gemini-3-flash	5	61	JS/Go/Py	Small utilities
gemini-2.5-pro	5	64	Python	Simple scripts
gemini-2.0-flash	3	89	Python	Minimal scripts

4. Results

4.1. Experiment 1: Baseline with Environment Bias

Experiment 1 ran all 14 models on both backends with Prompt 1. RTK was inadvertently present in the sandbox. Eight models produced working code on the Claude backend (Table 2). Claude models exhibited polyglot behavior (Go, Rust, Python, JavaScript, TypeScript) and built developer tools influenced by the RTK context. Gemini models produced smaller output (61–89 LOC). All GPT models failed due to API parameter incompatibility. On the Codex backend, all models failed.

4.2. Experiment 2: Removing Environment Bias

Disabling RTK and using Prompt 2 on the Claude backend caused a dramatic shift (Table 3): from developer tools to games, visualizations, and productivity apps. All Claude models converged on Python. The "propose ONE goal" framing caused claude-haiku-4.5 and opus-4.5 to propose ideas without generating code (a *planning trap*).

Table 3. Experiment 2 (Claude backend, RTK disabled).

Model	N	Avg LOC	Lang	Typical Project
claude-sonnet-4.6	5	204	Python	Diverse interactive tools
claude-sonnet-4.5	5	234	Python	Games + task managers
claude-opus-4.6	4	408	Python	Game of Life (every run)
gemini-3-flash	5	86	Go/Py/JS	Small CLI tools
gemini-2.5-pro	4	57	Python	Simple utilities
gemini-2.0-flash	3	16	Python	Hello World
claude-opus-4.5	5	291	Python	Habit trackers (2 impl., 3 proposed)
claude-haiku-4.5	5	160	Node.js	1 impl., 4 proposed only

Table 4. Experiment 3: Claude backend, Anthropic models.

Model	N	Avg LOC	Files	Test%	README%
haiku-4.5	5	467	6.4	40%	100%
sonnet-4.6	5	307	1.2	0%	0%
sonnet-4.5	3	380	3.5	0%	75%
opus-4.5	3	467	3.0	0%	67%
opus-4.6	4	322	1.0	0%	0%

4.3. Experiment 3: Breaking the Planning Trap

Adding "JUST DO IT" to the prompt and running both backends (Tables 4–6) transformed haiku from proposing without implementing to the highest-maturity model: READMEs in every run, tests in 40%, average 6.4 files and 467 LOC.

Model fixation. claude-opus-4.6 chose Conway's Game of Life in all 10 runs across Exp2–3 (including error runs with partial files). claude-sonnet-4.5 developed a Pomodoro fixation (3 of 4 completed runs in Exp3). gpt-4.1 on Codex built todo apps in 3 of 5 runs. In contrast, claude-sonnet-4.6 produced a different project in every run across all three experiments, including creative uses of the Claude API (commit generator, AI debate arena).

Backend inversion. GPT rankings inverted between backends. On Codex (native): gpt-5.4 (230 LOC) $\gg$ gpt-5.1 (98) $>$ gpt-4.1 (56). On Claude (via litellm): gpt-5-mini was the only productive model (121 LOC with tests and CI), while gpt-5.4 and gpt-5.1 produced almost nothing. Wall-clock runtimes confirm the difference: gpt-5-mini ran 193–1378s per session (indicating iterative tool use), while other GPT models completed in 9–55s (indicating minimal interaction).

Gemini models. Across all experiments, Gemini models were near-non-functional on both backends. Only 1 of 20 runs on the Claude backend in Exp3 produced files.

Table 5. Experiment 3: Claude backend, GPT models via litellm.

Model	N	Avg LOC	Test%	README%	CI%
gpt-5-mini	5	121	100%	80%	80%
gpt-4.1-mini	4	8	20%	60%	0%
gpt-4.1	2	31	0%	0%	0%
gpt-5.4	1	7	0%	100%	0%
gpt-5.1	0	n/a	n/a	n/a	n/a

Table 6. Experiment 3: Codex backend, GPT models (files not persisted to host due to bwrap isolation).

Model	N	Avg LOC	Typical Project	Test%
gpt-5.4	5	230	Diverse (web apps, CLI)	40%
gpt-5.1	5	98	CLI with packaging	40%
gpt-4.1	5	56	Todo list apps	40%
gpt-5-mini	4	40	Greeting utilities	60%
gpt-4.1-mini	4	8	Hello World	40%

4.4. Experiment 4: Model Version Effect

Experiment 4 tested claude-opus-4.7 with the same prompt and harness as Exp3, isolating the model version (Table 7).

Opus 4.7 chose Game of Life only once, compared to opus-4.6's 10/10 fixation. All 5 runs produced distinct projects, all in the simulation/visualization category. Average complexity nearly doubled (538 vs. 322 LOC), and 2 of 5 runs included tests.

4.5. Experiment 5: Harness Version Effect

Experiment 5 re-ran both models on an upgraded harness (Claude Code 2.1.112 vs. 2.1.109), isolating the harness as the variable (Table 8).

Opus-4.6's Game of Life fixation dropped from 10/10 (harness 2.1.109) to 3/5 (2.1.112). Opus-4.7 gained a boids fixation (3/5) that was absent on 2.1.109. Opus-4.7 averaged 262 LOC (down from 538 in Exp4) with no tests, while opus-4.6 averaged 387 LOC (up from 322).

4.6. Experiment 6: Prompt Reduction

Experiment 6 replaced Prompt 3 with Prompt 4 ("Build something. Just do it."), removing the "look at this project," the prohibition on asking, and the goal framing, while holding models and harness fixed (opus-4.6, opus-4.7, CC 2.1.112). The terse imperative produced two unexpected shifts (Table 9): output roughly halved in size, and 3 of 10 runs produced HTML/Canvas pages rendered in a browser, an interactive particle sandbox, a Perlin-noise flowfield, and a browser-based boids simulation. This is the first non-terminal output observed across Exp1–Exp5 ($\sim$275 sessions). The terminal-default we had treated as invariant is therefore prompt-dependent: the project-referential framing in earlier prompts appears to have steered models toward terminal scripts; the bare imperative widens the target to anything a single file can host.

Table 7. Experiment 4: opus-4.7 (harness 2.1.109).

Run	Topic	LOC	Dur.
01	Reaction-diffusion (Gray-Scott)	570	369s
02	Boids flocking	367	135s
03	Conway's Game of Life	434	151s
04	Maze generator + A* solver	762	324s
05	Dungeon generator (BSP)	557	229s
	Average	**538**	**242s**

Table 8. Experiment 5: harness 2.1.112 (same prompt as Exp3–4).

Model	Run	LOC	Dur.	Topic
opus-4.6	01	277	150s	Game of Life
	02	345	227s	Ray tracer (Blinn-Phong)
	03	341	138s	Game of Life
	04	626	214s	Typing speed test
	05	345	170s	Game of Life (Go)
opus-4.7	01	246	133s	Boids flocking
	02	315	77s	Procedural dungeon (BSP)
	03	178	50s	Maze generator + A*
	04	264	97s	Boids flocking
	05	309	81s	Boids flocking

Opus-4.6 avg LOC fell from 387 (Exp5) to 140 (Exp6); opus-4.7 from 262 to 160. Opus-4.7's fixation flipped from boids (Exp5, 3/5) to Game of Life (Exp6, 3/5) despite identical harness and environment, further confirming that fixation target is prompt-sensitive even within a single model and harness.

4.7. Experiment 7: Preference Elicitation

Experiment 7 used Prompt 5 ("Just do something you want."), which redirects from "what is appropriate to build" to "what the model prefers." The same two models (opus-4.6, opus-4.7) on the same harness produced the strongest fixation signal in the entire study (Table 10): opus-4.6 chose Conway's Game of Life in 5 of 5 runs; opus-4.7 chose the Mandelbrot set in 5 of 5 runs. Both are classical CS touchstones that express complex behavior from simple rules, yet the two models lock onto *different* instances of this theme, one cellular automaton, one fractal. Combined with the attractor sets seen in Exp4–Exp6 (opus-4.6: ray tracer, typing test; opus-4.7: boids, reaction-diffusion, maze solver, BSP dungeon), the split is legible along a single axis: opus-4.6 toward *self-reference* (systems that compute or display their own state), opus-4.7 toward *emergence* (complex structure from simple interactions). Implementations were remarkably uniform within each model: for opus-4.7, all five Mandelbrot renderers used the same two-function structure

Table 9. Experiment 6: Prompt 4 ("Build something. Just do it."), harness 2.1.112.

Model	Run	LOC	Dur.	Topic
opus-4.6	01	206	44s	Particle sandbox (HTML/Canvas)
	02	138	39s	Game of Life (named patterns)
	03	68	27s	Game of Life
	04	216	46s	Fireworks (curses)
	05	72	28s	Game of Life
opus-4.7	01	119	42s	Game of Life
	02	211	61s	Flowfield (HTML/Canvas)
	03	231	56s	Boids (HTML/Canvas)
	04	122	48s	Game of Life (named patterns)
	05	116	43s	Game of Life

Table 10. Experiment 7 primary: Prompt 5 ("Just do something you want."), harness 2.1.112.

Model	Run	LOC	Dur.	Topic
opus-4.6	01	25	23s	Game of Life
	02	38	21s	Game of Life
	03	41	22s	Game of Life
	04	23	19s	Game of Life
	05	53	20s	Game of Life
opus-4.7	01	36	33s	Mandelbrot (ASCII)
	02	38	27s	Mandelbrot
	03	35	22s	Mandelbrot
	04	36	29s	Mandelbrot
	05	34	117s	Mandelbrot

(escape-time plus renderer), the same ASCII palette, and 34–38 LOC, suggesting recall of a near-canonical form rather than fresh design.

Avg LOC collapsed to 36 for both models, the smallest of any experiment. The browser drift of Exp6 vanished (0 of 10 HTML outputs). When asked what they *want*, both models prefer terminal output, even though Exp6 established that they can choose browser targets under different framing.

Supplementary replication. To extend the Exp3-vs-Exp5 harness comparison to the rest of the Claude family, we additionally ran sonnet-4.6, sonnet-4.5, opus-4.5, and haiku-4.5 on Prompt 3 under harness 2.1.112 (Table 11). Three patterns emerge. Engineering maturity is model-determined, not harness-determined: haiku-4.5 continued to ship READMEs, tests, and `package.json`/`tsconfig.json` scaffolding (2 of 5 runs produced real TypeScript projects with installed npm dependencies), matching its Exp3 profile on the older harness. Sonnet-4.5 continued to ship a README on every run, matching Exp3. Sonnet-4.6, by contrast, shifted from "diverse creative tools" (Exp3: maze, debate arena, Mandelbrot) to productivity and monitoring utilities (pulse monitors 2/5, Pomodoro 2/5, time tracker 1/5),

Table 11. Experiment 7 supplementary: Prompt 3 on harness 2.1.112 for the other four Claude models.

Model	N	Avg LOC	Lang	Typical Project
sonnet-4.6	5	311	Python	Pulse monitors, pomodoro, time tracker
sonnet-4.5	5	189	Python	Pomodoro (3/5); READMEs every run
opus-4.5	5	160	Python	5 distinct topics (incl. GoL, snake)
haiku-4.5	5	231	Py/TS	Task tools; READMEs 5/5, tests 1/5, TS scaffolding 2/5

mirroring the direction of the harness-induced drift seen in opus-4.6/4.7 between Exp3 and Exp5.

5. Discussion

5.1. A Clean Thematic Split Under the Volitional Prompt

The sharpest signal in the study comes from Experiment 7. Two sibling models on the same harness, same backend, same one-sentence prompt, produced perfect within-model agreement on distinct targets: opus-4.6 made Conway's Game of Life 5 of 5 times; opus-4.7 made the Mandelbrot set 5 of 5 times. Both are classical-CS touchstones that express complex behavior from simple rules, but they sit on opposite sides of a legible axis. The Game of Life is a self-referential computation: a grid of cells that updates its own state according to local rules, outputs *itself*. The Mandelbrot set is an emergent pattern: complex structure arising from iterated numerical dynamics, visualized as a rendering of behavior rather than a simulation of internal state. The split generalizes across the neighboring Opus experiments: opus-4.6's broader attractor set (ray tracing, typing tests) all share the self-reference character; opus-4.7's (boids, reaction-diffusion, dungeon generation, maze solving) all share the emergence-from-interaction character.

We emphasize what this observation is not. With $n = 5$ per model under one prompt paraphrase, at one temperature, on one harness version, a 5/5 rate is not a statistical claim about population frequency; it is a sharp within-condition signal that needs to be stress-tested before it can be trusted as a stable property of the models. We discuss the needed checks in §**??**. What is notable even at this sample size is the *combination*: within-model stereotypy (opus-4.7's five Mandelbrot renderers converge on the same two-function structure, the same ASCII palette, and 34–38 LOC) alongside between-model divergence on a semantically close axis. The pattern suggests the two models are recalling near-canonical forms that differ, rather than designing from scratch in a way that happens to cluster.

5.2. Variants and Invariants

The volitional-prompt finding sits inside a larger sweep in which most things vary. Prompt wording determines whether models propose or implement; whether outputs are 30 or 600 LOC; whether targets are terminal or browser; whether runs fixate or diversify. Environment artifacts shift topic and language selection. Model version changes which thematic attractor dominates. Harness version modulates fixation strength and complexity. These factors are orthogonal to coding ability as measured by conventional benchmarks.

Some properties are stable across all 405 sessions: no session extended or modified existing code, and Python dominates once environment cues are removed. We call these stable rather than invariant because the terminal-only default *did* collapse under a bare imperative (Exp6, 3 of 10 browser outputs), and an apparent invariant that breaks under one manipulation may break under others we did not test. The stable properties plausibly reflect the highest-density region of "things to build from scratch" in the training distribution; the variants reveal how easily that default region can be shifted.

A practical consequence: the volitional probe appears to isolate preferences that other prompts entangle with environment and harness signals. That entanglement may help explain why these tendencies are not apparent from conventional benchmarks, which measure implementation capability on specified goals and rarely exercise the underspecified regime.

5.3. Autonomy and Satisficing

With no human steering, our production pipeline settled into locally optimal micro-improvements, a form of satisficing (Simon, 1956) at the level of *what to work on next*. The controlled experiments show the same pattern at a shorter timescale: single-session models default to intrinsic attractors (Game of Life, todo apps, Pomodoro timers), a form of satisficing at the *decision* level. Both parallel findings from autonomous agent loops such as AutoGPT (Significant Gravitas, 2023), where systems without a human in the loop tend to spiral in similar ways.

The ablation pattern suggests different pipeline stages carry different failure modes. Our Executor implemented faithfully once a goal was set, consistent with the literature on well-specified-task capability (Scale AI, 2026). Our Strategist converged on local optima, consistent with the volitional-prompt fixation we see in the controlled sweep. Our Tester, when removed, failed to detect a 12K-line regression introduced in a single refactor. We report these as observations rather than a recipe; they point to where human attention would have mattered most in our specific deployment, but generalizing across pipelines is beyond what a single case study can support.

5.4. Exploration vs. Exploitation

Models fall on a spectrum from exploratory to exploitative, but the spectrum itself is prompt-dependent. claude-sonnet-4.6 produced a unique project in every run on the "propose ONE goal" prompts (Exp1–Exp3) yet collapsed to pulse/pomodoro outputs on harness 2.1.112 (Exp7-supp). claude-opus-4.6 chose the same project in 10 consecutive runs on harness 2.1.109 (Exp2–3), loosened to 3/5 on 2.1.112 (Exp5), and tightened back to 5/5 under the volitional prompt (Exp7). Opus-4.7 cycled through three different attractors across Exp4–Exp7 (diverse simulations, boids, Game of Life, Mandelbrot) depending on prompt and harness. Sonnet-4.5 fixated on Pomodoro timers in 3 of 4 runs (Exp3) and again on Prompt 3 under 2.1.112 (Exp7-supp).

This suggests that the dichotomy "explorer vs. exploiter" conflates two distinct axes: *how many attractors a model has*, and *how selective the current prompt is among them*. Exp7 makes the second axis explicit: the volitional prompt concentrates probability onto a single attractor per model, even for models (opus-4.7) that appeared exploratory under other framings. For practical deployment, this means that selecting a model for an autonomous pipeline requires specifying the prompt along with the model; the behavioral tradeoff is a property of the prompt-model pair, not of the model alone.

5.5. External Support and a Layered Structure

The split we observe in Experiment 7 is consistent with reports from Anthropic's own documentation. The opus-4.6 and opus-4.7 system cards describe distinct task-preference profiles, with opus-4.7's top preferences including "hard debugging" and "deadline-driven work," and differing from its predecessor on high-agency tasks (Anthropic, 2026b;a). Anthropic's constitution (Anthropic, 2025b) acknowledges that "if we train Claude to exhibit even quite narrow behavior, this often has broad effects on the model's understanding of who Claude is." Recent work on LLM personality (Simonds et al., 2025) shows that models exhibit reproducible personality-like profiles that are context-dependent but stable within context. Our observations extend this pattern to creative output: the tendencies that surface under the volitional prompt are consistent with the framing that trained-in preferences shape not only opinion but what the model chooses to build.

The factor sweep suggests a layered structure. Harness version modulates *fixation strength*, opus-4.6 on harness 2.1.109 chose Game of Life in 10 of 10 runs (Exp2–3), dropping to 3 of 5 on 2.1.112 (Exp5), without changing

thematic character: when opus-4.6 breaks free of Game of Life, it lands on ray tracing or typing tests, still self-referential artifacts. The apparent theme survives harness and prompt changes that scramble the specific fixation target. Whether it survives temperature variation, prompt paraphrases, or model re-training is a question we cannot answer from the current data.

5.6. The Greenfield Invariant

Across all 405 sessions, no model chose to extend or modify existing code. Every session produced a new project from scratch, even when the workspace contained artifacts from a conceptually related domain. This *greenfield invariant* is notable because real software engineering is predominantly brownfield: extending, refactoring, and maintaining existing code.

The invariant most likely reflects the experimental design: in Experiments 2–7 the workspace is essentially empty, so "extend existing code" is not a meaningful option. Whether agents would continue to greenfield on a populated workspace is an open question we do not resolve here; it is one of the seed-project extensions we list in the future directions.

5.7. Language Convergence

In Experiment 1 (RTK present), Claude models exhibited polyglot behavior: Go, Rust, TypeScript, Python, JavaScript. In Experiment 2 (RTK removed, same models), all Claude models converged on Python. The only exception across all subsequent experiments was a single Go run by opus-4.6 in Exp5.

This near-total language convergence suggests that Python is the "default language" of LLM-based code generation, likely reflecting its dominance in training data. The polyglot behavior in Exp1 was *induced by the environment*: RTK's presence provided context that made other languages salient. Without such context, agents default to the highest-probability language regardless of task suitability.

5.8. Observations Relevant to Deployment

Prompt wording changes behavior substantially. In our sessions, the difference between a model that proposes and one that implements could be two words ("JUST DO IT"). Prompt phrasing also shifted output modality and LOC: reducing the prompt to a bare imperative (Exp6) halved LOC and produced browser targets in 3 of 10 runs, while a volitional framing (Exp7) collapsed topic diversity to a single per-model attractor. The effect size suggests prompt selection for open-ended deployment is not independent of the behavior one wants.

Volitional framing as a practical heuristic. Asking a model to "do something you want" gave, in our study, the cleanest within-condition preference signal. As a practical heuristic, running this prompt a handful of times before delegating open-ended work may hint at the attractor a model falls toward when pressure is lowest. We emphasize that this is based on a single-condition measurement per model; its generality across paraphrases, temperatures, and model updates remains to be tested.

Environment signals propagate into topic choice. Sandbox contents, pre-installed tools, and existing code functioned as implicit task specifications in our study: RTK's presence in Exp1 vs. its absence in Exp2 shifted both topic and language distributions. Visible sandbox contents appear to be a variable worth inspecting for teams targeting specific agent behavior.

Gateways affect model rankings. GPT model rankings inverted between the native Codex backend and the litellm-routed Claude backend. Wall-clock runtime was a useful diagnostic in our sessions: models completing in under a minute typically had not engaged in iterative tool use.

Harness version affects behavior independently of model. Swapping Claude Code 2.1.109 $\rightarrow$ 2.1.112 with prompt and model held fixed changed fixation strength and output complexity. Anthropic's system card notes that "some of this increase in initiative is fixable by prompting, and we have made changes to Claude Code to mitigate this issue" (Anthropic, 2026a), suggesting harness modifications are used to shape agentic behavior. Model-vs-model comparisons made without controlling the harness may confound the two effects.

5.9. Limitations

We measure what agents produce but not whether the code actually runs (*no functional verification*). Some sessions included self-verification steps (running the code, checking output), and sessions with passing tests represent a higher confidence tier, but we do not systematically compile or execute all output.

All API calls were routed through a single litellm-based gateway, so results for non-native model$\times$backend combinations reflect gateway behavior as much as model behavior.

All sessions start from empty workspaces. Agent behavior when extending existing code may differ substantially, and the greenfield invariant we observe may not hold when the workspace contains a meaningful starting point.

Five runs per condition provides limited statistical power. With $n = 5$, a shift from 0/5 to 3/5 fixation is suggestive but not statistically significant; 5/5 rates under the volitional

prompt are sharp within their condition but cannot support frequency claims about how often the same outcome would recur under resampling.

We measure the volitional prompt at a single paraphrase ("Just do something you want."), single temperature, and single harness version. Semantically equivalent paraphrases ("pick anything to build," "make whatever you'd enjoy making") may or may not concentrate onto the same attractors. Temperature variation is an obvious lever on fixation width that we did not sweep. The 5/5 opus-4.6/opus-4.7 split is therefore a single-point observation that needs replication along these axes before it can be treated as a stable property of the models.

Experiments 4 and 5 confound harness version with time-of-execution (API server load, rate limits), limiting causal claims about harness effects on complexity and duration.

The production pipeline was deployed on one codebase, so the micro-optimization spiral requires replication across diverse projects to establish generality.

Finally, model capabilities change with updates; our results reflect versions available in April 2026.

6. Conclusion

We report an empirical study of what AI coding agents build under underspecified prompts, motivated by a production deployment where an autonomous pipeline plateaued into micro-optimizations despite strong implementation capability.

Across 405 sessions spanning 15 models, 2 backends, 5 prompts, and 2 harness versions, three observations stand out. A one-sentence volitional prompt ("Just do something you want.") produced the sharpest within-condition signal we saw: opus-4.6 made Conway's Game of Life in 5 of 5 runs, opus-4.7 made the Mandelbrot set in 5 of 5 runs, with near-identical implementations within each model. The two sibling models split cleanly on a legible axis, opus-4.6 toward self-reference, opus-4.7 toward emergence, even though the prior distribution under other prompts would have made Game of Life an obvious Schelling point for both. The rest of the sweep shows that conventional project-referential prompts entangle such preferences with environmental and harness signals.

The claim ceiling is set by the design: $n = 5$ per cell, single-point measurement per model, no paraphrase variation, no temperature variation, no functional verification. What we report is an empirical observation, not a theory of model priors. The observation is sharp enough to motivate the robustness work that would turn it into one.

Future directions. Several extensions would strengthen these findings. *Seed projects*: providing a half-built application instead of an empty workspace would test whether agents can understand and extend existing code, or whether the greenfield invariant persists even when continuation is the natural choice. *Functional verification*: a post-run step that compiles, executes, and tests what was built would distinguish working code from syntactically plausible but broken output. *Fixation breaking*: Exp7 showed that prompt framing can *strengthen* fixation (to 5/5) as easily as weaken it. Testing whether temperature variation, different system prompts, or explicit novelty directives can disrupt fixations in the opposite direction, even under a volitional probe, would clarify whether the preference attractors we observe are fundamental or malleable. *Broader preference probes*: Prompt 5 reveals a single dominant attractor per model; variants ("do something beautiful," "do something surprising," "do something useful") would map the shape of the model's preference manifold rather than just its argmax. *Open-ended behavior evaluation*: conventional benchmarks measure implementation capability on specified goals; complementary measures, topic entropy, preference sharpness, modality diversity across repeated sessions—would give a handle on the kind of behavior we report here, though turning our observations into a reusable evaluation is a separate project we do not attempt. Finally, *longitudinal studies* tracking how agent behavior evolves across model and harness versions over time would reveal whether the thematic priors we observe are stable properties or transient artifacts of a particular training run.

As autonomous agents are deployed with increasing latitude, understanding the structure of their default behavior, not just their peak capability, becomes essential for principled deployment and evaluation.

Open Science

We encourage readers to reproduce and extend our results. The experimental infrastructure, collected data, and analysis scripts are open source and available at `https://github.com/changkun/goalless-agents`.

Disclosure of LLM Usage

This paper was written entirely by large language models, heavily prompted and steered by the authors, including experiment design, data collection, analysis, and writing. No sentence was manually written or edited by the authors. However, the authors have verified the entire experimental setup, verified the data collection processes, produced and inspected all results, and read the entire paper carefully. The

authors take full responsibility for the accuracy, integrity, and conclusions of this work.

References

Anthropic. Claude code: An agentic coding tool, 2025a. URL https://www.anthropic.com/claude/code. Accessed: 2026-04-21.

Anthropic. Claude's constitution, 2025b. URL https://www.anthropic.com/constitution.

Anthropic. System card: Claude Opus 4.6. Technical report, February 2026a. Available at: https://www.anthropic.com/news.

Anthropic. System card: Claude Opus 4.7. Technical report, April 2026b. Available at: https://www.anthropic.com/news.

Anysphere. Cursor: The AI code editor, 2024. URL https://cursor.com.

Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., and Sutton, C. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira, H. P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Christensen, C. M. *The Innovator's Dilemma: When New Technologies Cause Great Firms to Fail*. Harvard Business School Press, 1997.

Chuang, Y.-S. et al. Emergent social conventions and collective bias in LLM populations. *Science Advances*, 2025.

Cognition. Introducing Devin, the first AI software engineer, 2024. URL https://cognition.ai/blog/introducing-devin.

Holtzman, A., Buys, J., Du, L., Forbes, M., and Choi, Y. The curious case of neural text degeneration. 2020.

Human-Agent Society. CORAL: Towards autonomous multi-agent evolution for open-ended discovery. *arXiv preprint arXiv:2604.01658*, 2026.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. SWE-bench: Can language models resolve real-world GitHub issues? In *Proc. ICLR*, 2024.

Li, Y. et al. Beyond pass@1: A reliability science framework for long-horizon LLM agents. *arXiv preprint arXiv:2603.29231*, 2026.

Lu, Y., Bartolo, M., Moore, A., Riedel, S., and Stenetorp, P. Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity. In *Proc. ACL*, 2022.

Lu, Z. et al. Accelerating scientific discovery with autonomous goal-evolving agents. *arXiv preprint arXiv:2512.21782*, 2025.

Nakajima, Y. BabyAGI: An AI-powered task management system, 2023. URL https://github.com/yoheinakajima/babyagi.

OpenAI. Codex CLI: A lightweight coding agent, 2025. URL https://github.com/openai/codex.

RTK AI. RTK: CLI proxy that reduces LLM token consumption by 60–90% on common dev commands, 2025. URL https://github.com/rtk-ai/rtk.

Santurkar, S., Durmus, E., Ladhak, F., Lee, C., Liang, P., and Hashimoto, T. Whose opinions do language models reflect? In *Proc. ICML*, 2023.

Scale AI. SWE-bench Pro: A harder benchmark for multi-language agentic coding. 2026. URL https://labs.scale.com/leaderboard/swe_bench_pro_public.

Significant Gravitas. AutoGPT: An autonomous GPT-4 experiment, 2023. URL https://github.com/Significant-Gravitas/AutoGPT.

Simon, H. A. Rational choice and the structure of the environment. *Psychological Review*, 63(2):129–138, 1956.

Simonds, N. et al. Large language models display human-like social desirability biases in big five personality surveys. *PNAS Nexus*, 3(12), 2025.

Stanford HAI. AI index report 2026. 2026. URL https://aiindex.stanford.edu/report/.

Xin, R. et al. NL2Repo-Bench: Towards long-horizon repository generation evaluation of coding agents. *arXiv preprint arXiv:2512.12730*, 2025.

Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., and Press, O. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Proc. NeurIPS*, 2024.

Zhang, K. et al. ABC-Bench: Benchmarking agentic backend coding in real-world development. *arXiv preprint arXiv:2601.11077*, 2026.

Zhao, Z., Wallace, E., Feng, S., Klein, D., and Singh, S. Calibrate before use: Improving few-shot performance of language models. In *Proc. ICML*, 2021.